WARA-PS API Specifications

Consolidated technical reference

Source: https://api-docs.waraps.org/ | Retrieved: 2026-07-15 | Pages collected: 37

About this reference

This document consolidates the publicly available Markdown pages linked from the WARA-PS API documentation site. It preserves the published explanations, topic conventions, JSON examples, ROS examples, task commands, Task Specification Tree material, and service-communication material in one navigable Word document.

The source remains authoritative. This compilation does not add protocol behavior beyond the published pages; each source page is identified with its direct URL.

Contents

Agent Communication

Agent Communication

Source page: https://api-docs.waraps.org/agent_communication/agent_communication.md

Communication between agents and the user interface and services in WARA-PS is mainly done through a MQTT broker, which is also accessed by ROS agents through a ROS-MQTT Bridge (agent_communication/topics/ros_mqtt_bridge.md).

Following chapters will guide you through the fundamentals of that communication, including design descisions that have been made, how to write certain topics, what tasks are already defined in the Core System and what parameters and values they use to name a few.

MQTT stands for Message Queuing Telemetry Transport and is a lightweight open source messaging protocol, or set of rules, used for machine-to-machine communication in environments with limited bandwidth. The protocol uses a publish/subscribe communication pattern with a unit publishing data to a topic that others are subscribing to.

MAIN DESIGN PRINCIPLE:

Try to minimize work by creating extensions in a general way, but that also works with the existing system. This so smaller refactoring and changes need to be done in the existing system to adapt. However, this does **not** mean to disallow the general optimal solution simply to accomodate the existing system. But start with some hard coding or assumptions of default just to be able to test and close the loop.

Do not freeze unnecessarily, be AGILE!

Some design decisions:

• Agents/robots are to send regular heartbeat messages with some information on a topic which allows for registering the availability of the agent to the rest of the system.

• Communication requiring a response should be asynchronous

• UUID version 4 shall be used to identify a communication message

• Unreliable communication is handled by resending commands with the same UUID until we give up or a response has arrived.

QoS

Source page: https://api-docs.waraps.org/agent_communication/qos.md

When using MQTT as the network protocol for communication, there are three QoS (Quality of Service) levels; QoS 0, QoS 1 and QoS 2.

For the most part the messages sent in the Core Sysytem are sent with QoS 0. This level does not ensure that the message is delivered to the broker, but it is enough for most applications in the system since the information that is published from the agents/services almost always are published using a set rate (for example tree times per second).

The choice of using the QoS 0 keeps the amount of TCP/IP packets sent over the network to a minimum and thus requires a lower bandwidth.

If information is not published with a set rate, in the case where a value is only published once and never changed, the standard practice is then to use a "retained" tag for the message. This ensures that the message is saved to the broker, even when the connection to the broker fails. In these cases, be aware that the message is deleted from the broker when it is no longer of use, by publishing an empty string to the same topic.

Commands

Source page: https://api-docs.waraps.org/agent_communication/tasks/commands.md

Commands, are messages sent to level 2 and above agents commanding them to perform certain tasks. They are sent to a subtopic of the agent that the agent itself subscribe to, namely: **"UNIT/exec/command"**.

There are *four* types of **commands**, namely:

• **ping (agent_communication/tasks/ping_command.md)**

• **signal-task (agent_communication/tasks/signal_task.md)**

• **query-status**: return the status of the task (running, failed, success, unknown)

• **start-task (agent_communication/tasks/start_task.md)**

Standard operating protocol:

• task-uuid should be optionally sent in start-task and if not sent, a generated task-uuid will be returned in the response message.

• If a response has not arrived re-send the command with the same com-uuid.

• The receiver of commands has to keep track of the history of commands so it can answer if a command is unknown or finished when recieving the command "query-status".

• Use the changing agent-uuid when rebooting to detect that the agent has lost its history of commands.

The commands are sent in a command message which:

• Can optionally contain start and end time (more in later versions).

• If no start time or end time is given, just execute directly and as fast as possible but you do not need to optimize. Regular as fast as possible.

When an agent recievs a command it will send back a response as well as feedback to that message via publishing to the topics **"UNIT/exec/response"** respectively **"UNIT/exec/feedback"**.

Response message: - Returns agent-uuid, task-uuid, com-uuid, a string for failure reason and a response string field. Possible response string values are: "running", "failed"and "finished".

Feedback messages: - Update of the status of the task - Should contain agent-uuid, com-uuid, task-uuid and status. The possible status strings are:

• running

• failed

• aborted

• paused

• finished

• starting

• task specific strings

Task specific strings can be any other status message that makes sence in the context of the agents actions.

JSON feedback response example

```
{
   "agent-uuid": "8309d3ef-56fc-4ab0-86b0-9d9705ccf043",
   "com-uuid": "b3c90b08-622c-41b5-948c-80ca0abb49dc",
   "status": "running",
   "task-uuid": "49fb88b8-4d4a-4268-a69a-35e620a1a5f4"
}
```

ROS sending JSON feedback example

Code example Python:

```
def send_feedback(unit, task_uuid, status):
   msg = {}
   msg["com-uuid"] = str(uuid.uuid4())
   msg["agent-uuid"] = agent_uuid
   msg["task-uuid"] = task_uuid
   msg["status"] = status
   msgstr = json.dumps(msg, sort_keys=True, indent=4, separators=(',', ': '))
   # print(msgstr)
   topic = "exec/feedback"
   print("Send feedback on topic:", topic)
```

```
try:
    client = rospy.ServiceProxy('send_topic_msg_to_unit', SendTopicMsgToUnit)
    resp = client(unit.lstrip("/"), topic, msgstr)
    print("send_feedback send_topic_msg RESP:", resp)
except rospy.ServiceException as e:
print("Service call failed: %s"%e)
```

image-object-detection

Source page: https://api-docs.waraps.org/agent_communication/tasks/image_object_detection.md

Description: Run detection on an image input stream. Result is put out as a sensor. Send **$enough** to stop the current detection process. Also the result can be seen in an image stream whose address is found using sensors.

Tags: None

Task Parameters:

• **image-input-url (string):** If non empty the URL for the image stream to do the detections on.

• **image-input-ros-topic (string):** The ROS topic for the input image stream. if given it overrides the image-input-url

• **score (float64):** The minimum sore 0-1 to output a detection.

• **annotation (string):** The values: debug, standard

• **detections-sensor-topic (string):** The ROS topic name for the sensor. Given as full path or relative to the namespace. Example: sensor/detections

• **object-types (strings):** A list of the object types to detect.

• **wanted-detection-rate (int32):** The wanted detection rate.

• **salient-points-topic (string):** The ROS topic name for the sensor. Given as full path or relative to the namespace. Example: sensor/detections

• **salient-score (float64):** The minimum sore 0-1 to output a salient point.

JSON example

ROS example

land

Source page: https://api-docs.waraps.org/agent_communication/tasks/land.md

Description: Causes the platform to land.

Tags: vehicle

Task Parameters:

• **p (geopoint):** The position at which the platform should land, if land-at-current-position-flag is false.

• **land-at-current-position-flag (bool):** If this flag is true, land at the current position of the aircraft. Otherwise, land at the position given by p

• **z (float64):** The offset from the default landing altitude to use in simula tions. Default value is 0.0

• **heading (float64):** Specify this for fixed-wing platforms that require a heading in order to land. Helicopters may ignore this parameter.

look-at-bearing

Source page: https://api-docs.waraps.org/agent_communication/tasks/look_at_bearing.md

Description: This task causes the platform with a camera to point that camera using the specified bearing. The task is intended for cases where one should for example capture video, so rather than pointing once, it encapsulates a process where the camera is continuously adjusted. The task will only terminate when a signal **$enough** is sent to the task.

Tags: payload

Task Parameters:

• **bearing (float64):** Bearing angle in degree. Positive is to the right (clock wise). If absoulte bearing is used zero degre is north

• **use-absolute-bearing (bool):** If true then bearing angle is relative to north. Otherwise it is relative to the vehicles direction.

• **tilt (float64):** Tilt angle in degree. Positive is up.

• **tilt-body-relative (bool):** If true then the tilt angle is relative to the body of the vehicle. Otherwise it is relative to the world (horisontal plane). If this flag is false and the camera cannot be controlled relative to the horisontal plane than act like the parameter was true.

JSON example

```
{
  "com-uuid": "06 143d7a-ceae-4274-8ad5-7df5f75 157c8",
  "command": "start-task",
  "execution-unit": "dji0",
  "sender": "commander",
  "task": {
    "name": "look-at-bearing",
    "params": {
      "bearing": 45,
      "tilt": –45,
      "tilt-body-relative": false,
```

```
        "use_absoiute_bearing": false
      }
    },
    "task-uuid": "0f2ae2c5-6fc1-465d-b94b-c1adc25a587f"
}
```

ROS example

look-at-position

Source page: https://api-docs.waraps.org/agent_communication/tasks/look_at_position.md

Description: The agent will try to observe a position by moving the camera (it is only allowed to move the camera). It will try to observe it until it gets an **$enough** signal. It will send back status in the feedback message telling if the position is in view. Possible values for status relative to that: in-view-center, in-view, not-in-view.

Tags: None

Task Parameters:

• **geopoint (geopoint):** The position to observe.

• **named-position (string):** If defined look at this position (that can change). If the string start with / then it is the position of that unit that is used instead.

JSON example

```
{
    "com-uuid": "173ba99f-cb4e-4c7f-87d0-c4888b1f1e9d",
    "command": "start-task",
    "execution-unit": "dji0",
    "sender": "commander",
    "task": {
        "name": "look-at-position",
        "params": {
            "geopoint": {
                "altitude": 40.0,
                "latitude": 57.7611363,
                "longitude": 16.6805011,
                "rostype": "GeoPoint"
            }
        }
    },
    "task-uuid": "fded481d-f3a9-46c1-a44e-af205b0190d9"
}
```

ROS example

look-at-positions

Source page: https://api-docs.waraps.org/agent_communication/tasks/look_at_positions.md

Description: The agent will try to observe a position or a set of positions by moving the camera (it is only allowed to move the camera). It will try to observe it until it gets an **$enough** signal. It will send back status in the feedback message telling if the position is in view. Possible values for status relative to that: in-view-center, in-view, not-in-view. If more than one position is given as argument then the position nearest to the vehicle should be selected.

Tags: None

Task Parameters:

• **geopoints (geopoints):** The positions to observe.

• **named-positions (strings):** If defined look at this position (that can change). If the string start with / then it is the position of that unit that is used instead.

JSON example


```
{
  "com-uuid": "05b87b68-c361-43f7-b0b8-b1b3543ee908",
  "command": "start-task",
  "execution-unit": "dji0",
  "sender": "commander",
  "task": {
    "name": "look-at-positions",
    "params": {
      "geopoints": [
        {
          "altitude": 40.0,
          "latitude": 57.7611363,
          "longitude": 16.6805011,
          "rostype": "GeoPoint"
        },
        {
          "altitude": 40.0,
          "latitude": 57.7615541,
          "longitude": 16.6798574,
          "rostype": "GeoPoint"
        },
        {
          "altitude": 40.0,
          "latitude": 57.7615255,
          "longitude": 16.6784841,
          "rostype": "GeoPoint"
        },
```

```
            {
               "altitude": 40.0,
               "latitude": 57.7609703,
               "longitude": 16.6780335,
               "rostype": "GeoPoint"
            }
         ]
      }
   },
   "task-uuid": "67c9526d-4abd-49cb-8146-100eba998e04"
}
```

ROS example

move-path

Source page: https://api-docs.waraps.org/agent_communication/tasks/move_path.md

Description: This node moves the agent along a path. It is allowed to prepare for moving inside the node, for example doing take-off before flying. Also the specified altitudes is the minimal altitude, the flying should find positions higher if the position is occupied with obstacles.

Tags: vehicle

Task Parameters:

• **waypoints (geopoints):** The positions to move to

• **speed (string):** Qualitative speed level. Possible values are "fast", "standard" and "slow", and the meaning of these parameters is platform-dependent.

JSON example

```
{
   "com-uuid": "0bac359a-7b77-467b-95f9-bbaed320aadd",
   "command": "start-task",
   "execution-unit": "dji0",
   "sender": "commander",
   "task": {
      "name": "move-path",
      "params": {
         "speed": "standard",
         "waypoints": [
            {
               "altitude": 40.0,
               "latitude": 57.7611363,
               "longitude": 16.6805011,
               "rostype": "GeoPoint"
            },
            {
               "altitude": 40.0,
```

```
                "latitude": 57.7615541,
                "longitude": 16.6798574,
                "rostype": "GeoPoint"
            },
            {
                "altitude": 40.0,
                "latitude": 57.7615255,
                "longitude": 16.6784841,
                "rostype": "GeoPoint"
            },
            {
                "altitude": 40.0,
                "latitude": 57.7609703,
                "longitude": 16.6780335,
                "rostype": "GeoPoint"
            },
            {
                "altitude": 40.0,
                "latitude": 57.760398,
                "longitude": 16.6787201,
                "rostype": "GeoPoint"
            },
            {
                "altitude": 40.0,
                "latitude": 57.7604781,
                "longitude": 16.6799003,
                "rostype": "GeoPoint"
            }
        ]
    }
  },
  "task-uuid": "15c42e27-6c43-4291-bf10-d55 334 148e35"
}
```


ROS example

move-to

Source page: https://api-docs.waraps.org/agent_communication/tasks/move_to.md

Description: This node moves the agent to a specific position. It is allowed to prepare for moving inside the node, for example doing take-off before flying. Also the specified altitude is the minimal altitude, the flying should find positions higher if the position is occupied with obstacles.

Tags: vehicle

Task Parameters:

- **waypoint (geopoint):** The position to move to

- **speed (string):** Qualitative speed level. Possible values are "fast", "standard" and "slow", and the meaning of these parameters is platform-dependent.

JSON example

```
{
   "com-uuid": "193f5b87-2c88-495e-941e-80 182be4451b",
   "command": "start-task",
   "execution-unit": "dji0",
   "sender": "commander",
   "task": {
      "name": "move-to",
      "params": {
         "speed": "standard",
         "waypoint": {
            "altitude": 40.0,
            "latitude": 57.7611363,
            "longitude": 16.6805011,
            "rostype": "GeoPoint"
         }
      }
   },
   "task-uuid": "27ce3be0-972c-4130-b252-4e3f74 832 497"
}
```

ROS example

navigate

Source page: https://api-docs.waraps.org/agent_communication/tasks/navigate.md

Description: Navigate through or to a sequence of waypoints. Depending on parameter flags repeat the sequence or end in a holding pattern or just end. Currently there is just one speed, that will be changed to one start speed and one speed per waypoint.

Tags: None

Task Parameters:

• **waypoints (geopoints):** Way points to navigate through or to.

• **commanded-speed (float64s):** If the commanded-speed parameter is specified use the value in the list for commanded speed for the corresponding waypoint.

• **speed (strings):** Qualitative speed level. Possible values are "fast", "standard" and "slow", and the meaning of these parameters is platform-dependent.

• **continue-flag (bool):** If true and loop flag is false and hold flag is false then continue straight ahead after reaching the end waypoint.

• **loop-flag (bool):** If true the loop the waypoints.

• **hold-flag (bool):** If true then do a holding pattern after the waypoints are finished or the TST have been enough:ed.

• **straight-line-flag (bool):** If true then try to follow the line between waypoints. Not used from start position to first waypoint.

observe-position

Source page: https://api-docs.waraps.org/agent_communication/tasks/observe_position.md

Description: The agent will try to observe a given position. It will observe it until it gets an **$enough signal**.

Tags: None

Task Parameters:

• **geopoint (geopoint):** The position to observe.

• **speed (string):** Qualitative speed level. Possible values are "fast", "standard" and "slow", and the meaning of these parameters is platform-dependent.

• **distance (float64):** If given or if greater than 0.0 then use this radius for circling the observation position for vehicles that needs to move while observing the position. For vehicles that can hover uses this is the distance from which to observe.

• **height (float64):** If vehicles can hover this is the wanted height above the position to observe.

• **duration (int32):** Time to observe the position in seconds. If negative never stop and an $enough signal have to be sent to finish the TST execution.

JSON example


```
{
  "com-uuid": "e74ea2f3-db13-442d-a2ee-11c5baba0145",
  "command": "start-task",
  "execution-unit": "dji0",
  "sender": "commander",
  "task": {
    "name": "observe-position",
    "params": {
      "distance": 10.0,
      "duration": 20,
      "geopoint": [
        {
          "altitude": 40.0,
          "latitude": 57.7611363,
          "longitude": 16.6805011,
          "rostype": "GeoPoint"
        }
      ],
      "height": -1.0,
      "speed": [
```

```
        ]
      }
    },
    "task-uuid": "46eab756-e0e7-47e3-948b-ad78c795 295c"
}
```

ROS example

ping command

Source page: https://api-docs.waraps.org/agent_communication/tasks/ping_command.md

The "ping" task/command is used to check that an agent is active and ready to receive commands. A response message with "pong" as the response string should be sent by the agent when receiving this command.

JSON task example

```
{
    "com-uuid": "7300d049-176f-4dcb-b0bf-151 980d5611a",
    "command": "ping",
    "sender": "commander"
}
```

JSON response example

```
{
    "agent-uuid": "3887ced0-4e2d-4be4-aeff-6ba7 242 853a1",
    "com-uuid": "1c5284da-b803-49d1-8c39-b96ef00 603e7",
    "response": "pong",
    "response-to": "29c24bbf-88f0-4de2-97e9-4bd0d5f6bf23"
}
```

ROS sending JSON response example

Code example Python:

```
def send_response(unit, response_uuid, task_uuid, response, fail_reason=""):
    msg = {}
    msg["com-uuid"] = str(uuid.uuid4())
    msg["response-to"] = response_uuid
    msg["agent-uuid"] = agent_uuid
    msg["task-uuid"] = task_uuid
    msg["response"] = response
    msg["fail-reason"] = fail_reason
    msgstr = json.dumps(msg, sort_keys=True, indent=4, separators=(',', ': '))
    topic = "exec/response"
    print("send_response:", unit, topic, msgstr)
    try:
        client = rospy.ServiceProxy('send_topic_msg_to_unit', SendTopicMsgToUnit)
        resp = client(unit.lstrip("/"), topic, msgstr)
```

```
    print("send_response send_topic_msg RESP:", resp)
except rospy.ServiceException as e:
print("Service call failed: %s"%e)
```

search-area

Source page: https://api-docs.waraps.org/agent_communication/tasks/search_area.md

Description: This node specifies a search of an area given by a list of geopoints. The area is 'closed' by assuming a segment between the last and first position in the list of geopoints. It is allowed to prepare for moving inside the node, for example doing take-off before flying.

Tags: vehicle

Task Parameters:

- **area (geopoints):** The area to search.
- **speed (string):** Qualitative speed level. Possible values are "fast", "standard" and "slow", and the meaning of these parameters is platform-dependent.
- **target-type (string):** The type of the search target. Possible values: human, boat, ship, car, ...
- **area-type (string):** The type of the search target. Possible values: water, beach, forest, field
- **target-size (float64):** The estimates size of the search target in meter.

JSON example

```
{
  "com-uuid": "e16b95da-e3ef-4ba7-99f5-6db1680ad078",
  "command": "start-task",
  "execution-unit": "dji0",
  "sender": "commander",
  "task": {
    "name": "search-area",
    "params": {
      "area": [
        {
          "altitude": 40.0,
          "latitude": 57.7611363,
          "longitude": 16.6805011,
          "rostype": "GeoPoint"
        },
        {
          "altitude": 40.0,
          "latitude": 57.7615541,
          "longitude": 16.6798574,
          "rostype": "GeoPoint"
        },
        {
          "altitude": 40.0,
```

```
                "latitude": 57.7615255,
                "longitude": 16.6784841,
                "rostype": "GeoPoint"
            },
            {
                "altitude": 40.0,
                "latitude": 57.7609703,
                "longitude": 16.6780335,
                "rostype": "GeoPoint"
            }
        ],
        "speed": "standard",
        "target-size": 2.0
    }
  },
  "task-uuid": "dc5adc1f-eb01-40a9-be36-040b5e143e70"
}
```


ROS example

search-areas

Source page: https://api-docs.waraps.org/agent_communication/tasks/search_areas.md

Description: This node specifies a search of a set of area. The areas are "closed" by assuming a segment between the last and first position in the list of geopoints. It is allowed to prepare for moving inside the node, for example doing take-off before flying.

Tags: vehicle

Task Parameters:

• **areas (geopointarrays):** The areas to search.

• **speed (string):** Qualitative speed level. Possible values are "fast", "standard" and "slow", and the meaning of these parameters is platform-dependent.

• **target-type (string):** The type of the search target. Possible values: human, boat, ship, car, ...

• **area-type (string):** The type of the search area given as a list of tags. Possible tag values: water, beach, forest, field

• **target-size (float64):** The estimates size of the search target in meter.

JSON example


```
{
  "com-uuid": "c6929a0e-ad60-40d9-8f60-9b05d632c0fe",
  "command": "start-task",
  "execution-unit": "op0",
  "sender": "commander",
```

```
"task": {
    "name": "search-areas",
    "params": {
        "areas": [
            [
                {
                    "altitude": 40.0,
                    "latitude": 57.7611363,
                    "longitude": 16.6805011,
                    "rostype": "GeoPoint"
                },
                {
                    "altitude": 40.0,
                    "latitude": 57.7615541,
                    "longitude": 16.6798574,
                    "rostype": "GeoPoint"
                },
                {
                    "altitude": 40.0,
                    "latitude": 57.7615255,
                    "longitude": 16.6784841,
                    "rostype": "GeoPoint"
                },
                {
                    "altitude": 40.0,
                    "latitude": 57.7609703,
                    "longitude": 16.6780335,
                    "rostype": "GeoPoint"
                }
            ],
            [
                {
                    "altitude": 40.0,
                    "latitude": 57.7611363,
                    "longitude": 16.6805011,
                    "rostype": "GeoPoint"
                },
                {
                    "altitude": 40.0,
                    "latitude": 57.7615541,
                    "longitude": 16.6798574,
                    "rostype": "GeoPoint"
                },
                {
                    "altitude": 40.0,
                    "latitude": 57.7615255,
                    "longitude": 16.6784841,
                    "rostype": "GeoPoint"
                },
                {
                    "altitude": 40.0,
                    "latitude": 57.7609703,
                    "longitude": 16.6780335,
```

```
                "rostype": "GeoPoint"
            }
          ],
          [
            {
              "altitude": 40.0,
              "latitude": 57.7611363,
              "longitude": 16.6805011,
              "rostype": "GeoPoint"
            },
            {
              "altitude": 40.0,
              "latitude": 57.7615541,
              "longitude": 16.6798574,
              "rostype": "GeoPoint"
            },
            {
              "altitude": 40.0,
              "latitude": 57.7615255,
              "longitude": 16.6784841,
              "rostype": "GeoPoint"
            },
            {
              "altitude": 40.0,
              "latitude": 57.7609703,
              "longitude": 16.6780335,
              "rostype": "GeoPoint"
            }
          ]
        ],
        "speed": "standard",
        "target-size": 4.0
      }
    },
    "task-uuid": "3f800 056-7340-4607-8660-930 297adc8b7"
}
```


ROS example

search-around-position

Source page: https://api-docs.waraps.org/agent_communication/tasks/search_around_position.md

Description: This node specifies a search of an area given by a list of geopoints. It is allowed to prepare for moving inside the node, for example doing take-off before flying.

Tags: vehicle

Task Parameters:

- **geopoint (geopoint):** The area to search.

• **radius (float64):** The radius around the geopoint to focus on.

• **speed (string):** Qualitative speed level. Possible values are "fast", "standard" and "slow", and the meaning of these parameters is platform-dependent.

• **target-type (string):** The type of the search target. Possible values: human, boat, ship, car, ...

• **target-size (float64):** The estimates size of the search target in meters.

JSON example

```
{
  "com-uuid": "d37a2e04-c739-4691-a667-de8de8f3e7e8",
  "command": "start-task",
  "execution-unit": "dji0",
  "sender": "commander",
  "task": {
    "name": "search-around-position",
    "params": {
      "geopoint": {
        "altitude": 28.9,
        "latitude": 57.7611363,
        "longitude": 16.6805011,
        "rostype": "GeoPoint"
      },
      "radius": 25.0,
      "speed": "standard",
      "target-size": 2.0,
      "target-type": "human"
    }
  },
  "task-uuid": "f0dbb936-0c64-4b73-b2c5-85f718e1d7ba"
}
```

ROS example

signal-task

Source page: https://api-docs.waraps.org/agent_communication/tasks/signal_task.md

Each task/command activated via the command "start-task" should have built-in signals that can effect the performance of the task.

The built-in signals are:

• **$enough** - commands agent to stop it"s task. The sender no longer needs it done.

• **$pause** - commands agent to pause in perfoming its task.

• **$continue** - commands agent to continue with the current paused task.

• **$abort** - commands agent to stop it"s task. The sender has determined the task failed and is aborting it.

Signal names starting with **$** are system signals. Other names are specific for the task.

JSON task example

```
{
   "com-uuid": "7300d049-176f-4dcb-b0bf-151 980d5611a",
   "command": "signal-task",
   signal: $abort,
   "sender": "commander",
   "task-uuid": "9b4c701c-e8a2-4a92-9370-87c8f57e0dcd"
}
```

JSON response example

```
{
   "agent-uuid": "3887ced0-4e2d-4be4-aeff-6ba7 242 853a1",
   "com-uuid": "1c5284da-b803-49d1-8c39-b96ef00 603e7",
   "response": "ok",
   "response-to": "7300d049-176f-4dcb-b0bf-151 980d5611a"
}
```

start-task

Source page: https://api-docs.waraps.org/agent_communication/tasks/start_task.md

The "start-task" command is, as the name implies, used to command the agent to start a "task", specified in the commands "task" parameter. The "task" is in the form of a json object containing the specifics of the task such as its name and what parameters it operates from.

Although "start-task" is the command, as the task it contain can be any one from a multitude of different task objects, the task can be seen as the core command as it is that performance of the task that is truly of interest. The "start-task" syntax can bee seen as the standard way of forwarding a task.

start-task command syntax:

```
{
   "com-uuid": <uuid v4 of command>,
   "command": "start-task",
   "execution-unit": <name of unit to execute the command>,
   "sender": <name of sender of command>,
   "task": {
      "name": <task name>,
      "params": {
         <specifics of the task e.g. waypoint, speed ect.>
      }
   },
   "task-uuid": <uuid v4 of task>
}
```

In the following sections, common tasks along with their task parameters and examples will be listed.

Each task section will contain:

• **Description** - Describes what the task specifies the agent should do.

• **Tags** - States what type of agents are relevant for the command, standard is "vehicle" mening all agents with mobility can perform the task.

• **Task Parameters** - Specifies which task parameters will be included in the sent command and thus need to be managed by the agent in order for it to perform the command correctly.

take-off

Source page: https://api-docs.waraps.org/agent_communication/tasks/take_off.md

Description: This node ensures that the platform is in the air. If this is already true, nothing will happen. If the platform is on the ground, it will take off to a vehicle-specific altitude.

Tags: vehicle

Task Parameters:

• **height-above-takeoff (float64):** If specified, then climb to the specified altitude after taking off (the value is relative to the takeoff position). Otherwise, the altitude after take-off is unspecified and platform-specific.

• **path (geopoints):** Used in proposal and execution. A sequence of waypoints to fly through or to.

Basic Naming Conventions

Source page: https://api-docs.waraps.org/agent_communication/topics/basic_naming_conventions.md

In order to standardise the topic paths and make it easier to find subtopics and their values, a naming convention has been created that all who intend to use the Core System services should follow.

JSON

For users connecting dircetly to the MQTT broker, following naming conventions applies:

__Naming scheme:__ PREFIX1/PREFIX2/PREFIX3/PREFIX4/UNIT/#

__For sensors:__ PREFIX1/PREFIX2/PREFIX3/PREFIX4/UNIT/sensor/SENSOR

__For heartbeat/info messages:__ PREFIX1/PREFIX2/PREFIX3/PREFIX4/UNIT/XXX

>• **PREFIX1:** - The context/ system the unit is used in, can be used to have parallel experiments/demos. E.g: "waraps-demo-2021", "waraps", "aiics-testing", and so on.The standard to connect to Core System services is to use "waraps"

• **PREFIX2:** - The type, in this case "unit" to indicate that it is a agent/robot/system that have heartbeat. Other possible values are general and service but those ar maninly used by Core System services and not relevant in this example.

• **PREFIX3:** The domain of the unit menaing where it is (most) active, e.g. "surface", "air", "subsurface", "ground". Used to filter e.g. boats from drones.

• **PREFIX4:** The state of the agent, wether it is "real", virtual that ha the topic "simulation" or a "playback" of a recording of the agent.

• **UNIT:** Name of the agent e.g. "usvc2", "piraya0", "dji1", "airpelagosystem0" ect.

• **SENSOR:** Name or reference to real or virtual sensor. Examples: "position", "speed", "course", "heading", "battery_state".

Examples of topic paths:

- waraps/unit/air/simulation/dji0/heartbeat (agent_communication/topics/heartbeat.md)
- waraps/unit/air/simulation/dji0/sensor_info (agent_communication/topics/sensor_info.md)
- waraps/unit/air/simulation/dji0/direct_execution_info (agent_communication/topics/direct_execution_info.md)
- waraps/unit/air/simulation/dji0/tst_execution_info (agent_communication/topics/tst_execution_info.md)
- waraps/unit/air/simulation/dji0/sensor (agent_communication/topics/sensor.md)/position
- waraps/unit/air/simulation/dji0/sensor (agent_communication/topics/sensor.md)/spatial
- waraps/unit/air/simulation/dji0/sensor (agent_communication/topics/sensor.md)/speed
- waraps/unit/air/simulation/dji0/sensor (agent_communication/topics/sensor.md)/heading
- waraps/unit/air/simulation/dji0/sensor (agent_communication/topics/sensor.md)/course
- waraps/unit/air/simulation/dji0/sensor (agent_communication/topics/sensor.md)/camera_url
- waraps/unit/air/simulation/dji0/exec (agent_communication/tasks/tasks.md)/command
- waraps/unit/air/simulation/dji0/exec (agent_communication/tasks/tasks.md)/response
- waraps/unit/air/simulation/dji0/exec (agent_communication/tasks/tasks.md)/feedback

__Please Note__

- Topics starts with a name, e.g. waraps/ **not** /waraps/
- Topics have all **lowercase** and separates words with **underscore '_'**, e.g. *.../sensor/camera_data*

ROS

For sensor topics see Sensors ROS example

The heartbeat/info messages in string format are sent on:

n.advertise<lrs_msgs_common::TopicMsg>("/heartbeat_info", 10);

The definition of TopicMsg is:

string topic

string msg

So the topic used in the JSON/MQTT API is put on the msg also. That is so that it can be forwarded to the JSON/MQTT part of the system.

WORK IN PROGRESS, TO BE EXTENDED.

Direct Execution Info

Source page: https://api-docs.waraps.org/agent_communication/topics/direct_execution_info.md

The "direct_execution_info" message should be sent for all level 2 agents and above as it provides information on which tasks (agent_communication/tasks/tasks.md) an agent is capable of doing. It also works as a heartbeat for the direct execution level.

Publish the direct_execution_info message to:

TOPIC: UNIT/direct_execution_info

Message parameters

• **name:** The name of the agent

• **rate:** The expected rate the message is sent, written in Hz

• **type:** A type field indicates what type of message it is. In this case direct execution info.

• **stamp:** Epoch time stamp of when the message was sent, in double (time.time() in Python, seconds in double)

• **tasks-available:** List of tasks that are supported by the agent and for each task a list of signals it understands.

• **tasks-executing:** List of tasks agent is currently executing.


```
{
  "name": "piraya0",
  "rate": 1.0,
  "type": "DirectExecutionInfo",
  "stamp": 1 614 080 836.5142891,
  "tasks-available": [
    {
```

```
        "name": "navigate",
        "signals": [
          "$abort",
          "$enough",
          "$pause"
        ]
      },
      {
        "name": "look-at",
        "signals": [
          "$abort",
          "$enough"
        ]
      }
    ],
    "tasks-executing": [
      {
        "task-name": "navigate",
        "task-uuid": "b6161aee-a90f-4f5c-8a22-7d814 725a6e2"
      }
    ]
}
```


Heartbeat

Source page: https://api-docs.waraps.org/agent_communication/topics/heartbeat.md

The "heartbeat" message is the most fundamental message an agent should publish in order to be a part of the Core System. This as it is used to indicate the existance of an agent.

Publish the heartbeat message to:

TOPIC: .../UNIT/heartbeat

One of the parameters the heartbeat should contain is a list of the agents info topics, meaning the levels it is registered on.

E.g. A level 2 agent should in this list state both "sensor", as well as "direct_execution" to display how it publishes to the corresponding info subtopics.

Message parameters:

• **agent-type:** The domain the agent (primarily) belongs to (which also is a part of the topic pathway). Possible values are: "air", "ground", "surface", and "subsurface".

• **agent-uuid:** Unique id for the agent (uuid v4), should change when rebooting and losing state information.

• **levels:** A list of levels to register on. Possible values are: "sensor", "direct execution", "tst execution" and "delegation". This means the agent is intended to work on these levels but the info messages then

indicate if it is available on that level. For example when a boat is out of communication range it will not send info messages on sensor and direct execution level.

• **name:** The name of the agent (which also is a part of the topic pathway)

• **rate:** The expected rate the message is sent, written in Hz

• **stamp:** Epoch time stamp of when the message was sent, in double (time.time() in Python, seconds in double)

• **type:** A type field indicates what type of message it is. In this case heartbeat.

Example


```
{
    "agent-type": "surface",
    "agent-uuid": "b992 552b-90b0-4911-8707-d6f808 362bd2",
    "levels": [
        "sensor",
        "direct execution"
    ],
    "name": "piraya0",
    "rate": 1.0,
    "stamp": 1 614 080 475.1084013,
    "type": "HeartBeat"
}
```


L1 Topics

Source page: https://api-docs.waraps.org/agent_communication/topics/l1_topics.md

Topics to which agents/services publish and subscribe their information are cruical to the workings of MQTT. However, as there is nothing preventing agents/services from publishing to the same topic, it becomes important to standardize how topics should be constructed.

This to make it easier for the user interfaces and services to find the information they require in order to work, as well as limit the possibility of users interfering with each other by overwriting each others data.

Design decisions:

• One info message should be sent for each API level, but at a lower update rate than the general HeartBeat message.

• Use different heartbeat/info messages for each level and send them for all levels you want to register on.

• A ”heartbeat” message shall be sent to indicate the existence of the agent.

• The ”heartbeat” message shall contain a list of levels it is registered on.

• The ”info” messages should have an agent-uuid field that is changed each time the agent reboots. This so that we can detect that the agent has lost its history of commands.

The topics that are necessary for an L1 agent to work with most Core System Services are:

• heartbeat (agent_communication/topics/heartbeat.md) - All agents

• sensor_info (agent_communication/topics/sensor_info.md) - Level 1 and above agents

• sensor (agent_communication/topics/sensor.md) - Level 1 and above agents

To understand and use these topics correctly, you must first understand the basic naming conventions for the topic paths.

L2 Topics

Source page: https://api-docs.waraps.org/agent_communication/topics/l2_topics.md

For more information on what topics are see L1 Topics (l1_topics.md)

The topics that are necessary for an L2 agent to work with most Core System Services are:

• heartbeat (agent_communication/topics/heartbeat.md) - All agents

• sensor_info (agent_communication/topics/sensor_info.md) - Level 1 and above agents

• sensor (agent_communication/topics/sensor.md) - Level 1 and above agents

• direct_execution_info (agent_communication/topics/direct_execution_info.md) - Level 2 and above agents

L3-L4 Topics

Source page: https://api-docs.waraps.org/agent_communication/topics/l3_l4_topics.md

For more information on what topics are see L1 Topics (l1_topics.md)

The topics that are necessary for an L3-L4 agent to work with most Core System Services are:

• heartbeat (agent_communication/topics/heartbeat.md) - All agents

• sensor_info (agent_communication/topics/sensor_info.md) - Level 1 and above agents

• sensor (agent_communication/topics/sensor.md) - Level 1 and above agents

• direct_execution_info (agent_communication/topics/direct_execution_info.md) - Level 2 and above agents

• tst_execution_info (agent_communication/topics/tst_execution_info.md) - Level 3 and above agents

ROS-MQTT Bridge/Communication

Source page: https://api-docs.waraps.org/agent_communication/topics/ros_mqtt_bridge.md

If you want to plug in a robot using ROS you can talk with tommy.persson@liu.se for more details.

Main methods:

• *Plugin in C++ using the available libraries.* This will be available for people asking for this. It is not possible to describe meaningfully here. You need to check out some code and look at some examples. In the repo lrs examples there are code examples for this.

• *Use an action interface.* This is work in progress with SMARC. A first prototype exists. More documentation later.

• *Using the JSON messages provided on ROS topics via a MQTT Bridge.*

To get the sensor data to the MQTT broker and coded in JSON, a bridge program has to be used. The one available can be started with the following docker-compose.yml file:

```
---
version: '3.3'
services:
  json-api:
    image: gitlab.liu.se:5000/lrs/waraps_docker_images/lrs-json-api-bridge:latest
    command: roslaunch lrs_json_api_bridge json_api.launch mqtt_host:=broker.waraps.org mqtt_user:=mqtt mqtt_passwd:=Check mqtt_tls:=true mqtt_prefix1:="waraps" mqtt_prefix2:="unit" mqtt_prefix3:="ground" mqtt_prefix4:="real" ns:=${NS} json_api_unit:=true --wait
    network_mode: host
    security_opt:
      - "apparmor:unconfined"
    environment:
      - ROS_MASTER_URI=http://localhost:11311
      - ROS_IP=127.0.0.1
```

Do remember to **change the prefix flags to the correct value.** "NS" is the namespace for the robot. So for example: "/dji0", "/leo1", ...

Sensor

Source page: https://api-docs.waraps.org/agent_communication/topics/sensor.md

The "sensor" subtopic is where an agents different sensors publish their data to the MQTT. It should correspond to the list found in the "sensor_info" message's "sensor-data-provided" parameter.

Publish sensor messages under the following topic:

TOPIC: UNIT/sensor/SENSOR

Note that "SENSOR" is the name of the sensor e.g. "speed", "heading" ect.

Mandatory sensors:

Please note that the following sensors and values are **mandatory** for an level 2 agent:

• **position**

• **heading**

• **course**

• **speed**

JSON sensor examples

• waraps/unit/air/simulation/dji0/sensor/position: GeoPoint *= json-object consisting of 4 values namely* {latitude: float, longitude: float, altitude: float, type: "GeoPoint"*(string)* }

• waraps/unit/air/simulation/dji0/sensor/heading: float

• waraps/unit/air/simulation/dji0/sensor/course: float

• waraps/unit/air/simulation/dji0/sensor/speed: float

ROS sensor examples

• /dji0/sensor/position: geographic msgs/GeoPoint

• /dji0/sensor/heading: std msgs/Float32

• /dji0/sensor/course: std msgs/Float32

• /dji0/sensor/speed: std msgs/Float32

Sensor Info

Source page: https://api-docs.waraps.org/agent_communication/topics/sensor_info.md

The "sensor_info" message contains information about which sensors an agent has that are publishing data to the MQTT. The message should be sent for all level 1 and above agents and is a companion to the "sensor (agent_communication/topics/sensor.md)" subtopic. It also works as a heartbeat for the sensor level which is why it should contain a list of all sensors published under the "sensor" subtopic.

Publish the sensor_info message to:

TOPIC: UNIT/sensor_info

Message parameters

• **name:** The name of the agent

• **sensor-data-provided:** List of sensors the agent retains and sends data about. Should correspond with the messages published to the "sensor" subtopic.

• **rate:** The expected rate the message is sent, written in Hz

• **stamp:** Epoch time stamp of when the message was sent, in double (time.time() in Python, seconds in double)

• **type:** A type field indicates what type of message it is. In this case sensor info.

Note! The strings listed in **sensor-data-provided** is the string used in **sensor topic**. All sensor topics is under "UNIT/sensor/". E.g: piraya0/sensor/position.

Example

```
{
"name": "piraya0",
"rate": 1.0,
"sensor-data-provided": [
    "position",
    "speed",
    "executing_tasks"
],
"stamp": 1 614 181 737.0478,
"type": "SensorInfo"
}
```

TST Execution Info

Source page: https://api-docs.waraps.org/agent_communication/topics/tst_execution_info.md

The "TST-execution-info" message is published by level 3 agents to inform that they are capable of recieving start-TST commands.

TST (Task Specification Tree)

Source page: https://api-docs.waraps.org/agent_communication/tst/tst.md

TST stands for **Task Specification Tree** and ...

Link to LiU documentation about TST: https://gitlab.liu.se/lrs2/lrs_doc

With a TST one can specify many tasks and the order in which the tasks should be performed. For every task in the tree one can also speicify which agent should perform the task. This means that if an agent has the ability to recieve a TST through a "start-TST" command, the agent should also be able to delegate tasks to other agents.

In the Core System we refer to TST as either "**specified**" or "**unspecified**" with the difference being how much of the task informations are specified.

In the case of "specified" TSTs all the information about the mission such as which unit shall perform what task and how ect. are all specified when the mission is sent from the command interface. In the case of "unspecified" TSTs however, the information about the mission is more lacking. An agent that can handle specified trees is of level 3, while an agent that can handle an unspecified tree is of level 4.

For communication with TST:s to work, in addition to the ability to delegate tasks, the agent also needs to save a new uuid for every node in the tree. This so that the commander sending the "start-tst"

command can send signals to parts of the tree, or to the whole tree by signaling to the root node of the tree.

Note that this API is towards the TST/Delegation system.

• Execution of fully instantiated TST trees

• The Task execution is the same as for level 2 and used internally in the TST system.

• The communication with command and control is different.

• Fully instantiated TST trees (represented in JSON) are sent to the system and the result is: start of execution possibly on more than one platform or direct failure to start.

• Update of progress during execution including report of success or failure for each node in the tree.

• During execution abort/pause/continue/enough can be sent to specific nodes.

• We might have help command to send to all nodes in a tree.

Topics

• UNIT/tst/command

• UNIT/tst/response

• UNIT/tst/feedback

The commands are usually sent to op0. So the most common topics to use are:

• op0/tst/command

• op0/tst/response

• op0/tst/feedback

But it is possible to send it directly to an agent also but the agent might not be running so it is safer to send it to the op0.

TST Commands

Source page: https://api-docs.waraps.org/agent_communication/tst/tst_commands.md

A command that is added for a level 3 and above agents is the "start-tst" command.

For this level 3+ command, the same Standard Operating Protocol is used, the same response and feedback messages. See the section on Commands (agent_communication/tasks/commands.md)

Signal a TST Tree

Source page: https://api-docs.waraps.org/agent_communication/tst/tst_commands/signal_a_tst_tree.md

Send the signal to all TST nodes in the tree by using the uuid of the root node of the tree when sending the signal.

JSON task example

```
{
  "com-uuid": "2b01c8f9-437f-49d1-869b-cd5e3d3bb395",
  "command": "signal-tst-tree",
  "node-uuid": "ccb261a7-3446-4e7e-9aea-d9d57eaed793",
  "sender": "commander",
  "signal": "$abort"
}
```

JSON response example

```
{
  "com-uuid": "9dfde776-9d80-4fe9-9e34-32 761f5ed545",
  "response": "ok",
  "response-to": "2b01c8f9-437f-49d1-869b-cd5e3d3bb395"
}
```

signal-tst-node

Source page: https://api-docs.waraps.org/agent_communication/tst/tst_commands/signal_tst_node.md

This command is used to send a signal to a specific node in the tree. The node can hold children of its own and the signal will then affect the child nodes as well.

JSON task example

```
{
  "com-uuid": "2b01c8f9-437f-49d1-869b-cd5e3d3bb395",
  "command": "signal-tst-node",
  "node-uuid": "ccb261a7-3446-4e7e-9aea-d9d57eaed793",
  "sender": "commander",
  "signal": "$abort"
}
```

JSON response example

Example:

```
{
  "com-uuid": "9dfde776-9d80-4fe9-9e34-32 761f5ed545",
  "response": "ok",
  "response-to": "2b01c8f9-437f-49d1-869b-cd5e3d3bb395"
}
```

signal-unit

Source page: https://api-docs.waraps.org/agent_communication/tst/tst_commands/signal_unit.md

Send the signal to all running TST nodes on the unit.

JSON task example

```
{
  "com-uuid": "2b01c8f9-437f-49d1-869b-cd5e3d3bb395",
  "command": "signal-unit",
  "unit": "/dji21",
  "sender": "commander",
  "signal": "$abort"
}
```

JSON response example

```
{
  "com-uuid": "9dfde776-9d80-4fe9-9e34-32 761f5ed545",
  "response": "ok",
  "response-to": "2b01c8f9-437f-49d1-869b-cd5e3d3bb395"
}
```

start-tst

Source page: https://api-docs.waraps.org/agent_communication/tst/tst_commands/start_tst.md

The "start-tst" command is sent to an agent to start the delegation of the tasks included in a tree. The command contains the tst tree that the agent should delegate or participate in.

In the following example the command contains a TST tree which contains two "move-to" tasks that are to be delegated by "tst_delegator_name" to two different agents, specified by an alias A and B.

If the tasks were to be executed by specific agents, the alias A should be replaced by "/" and the name of the agent. For example "/drone1".

JSON task example

```
{
 "com-uuid": "7300d049-176f-4dcb-b0bf-151 980d5611a",
 "command": "start-tst",
 "receiver": "tst_delegator_name",
 "sender": "c2",
 "tst": {
  "common_params": {
   "execunit": "/tst_delegator_name",
   "node-uuid": "6e4e360f-7248-47ce-8490-9332afcb91df"
  },
  "name": "conc",
  "params": {},
  "children": [
   {
    "children": [],
    "common_params": {
```

```
      "execunit": "A"
    },
    "name": "move-to",
    "meta": {
      "reference": "no_go_beach_A"
    },
    "params": {
      "target-type": "boat",
      "target-size": 2,
      "speed": "fast",
      "waypoint":
       {
         "latitude": 57.761467 232 936 354,
         "longitude": 16.680135 220 525 187,
         "altitude": 0,
         "rostype": "GeoPoint"
       }
    }
   },
   {
    "children": [],
    "common_params": {
      "execunit": "B"
    },
    "name": "move-to",
    "meta": {
      "reference": "nogo__zone-A"
    },
    "params": {
      "target-type": "person",
      "target-size": 2,
      "speed": "fast",
      "waypoint":
       {
         "latitude": 57.76092 056 857 479,
         "longitude": 16.681076 220 107 087,
         "altitude": 0,
         "rostype": "GeoPoint"
       }
    }
   }
  ]
 }
}
```


JSON response example


```
{
  "root-uuid": "fe1fbcee-1cae-4a69-b9f7-83 914 454a7a7",
  "com-uuid": "1c5284da-b803-49d1-8c39-b96ef00 603e7",
  "response": "running",
  "fail-reason": "",
  "response-to": "7300d049-176f-4dcb-b0bf-151 980d5611a"
```

```
}
```

TST Feedback messages

Source page: https://api-docs.waraps.org/agent_communication/tst/tst_feedback_messages.md

The possible values of the status is:

- not-active
- active
- waiting
- running
- failed
- revoked
- paused
- aborted
- succeeded

Example 1:

```
{
  "com-uuid": "ed3717a6-ffef-4129-ad21-ea577 117c706",
  "fail-reason": "",
  "node-uuid": "ff200 942-73a2-401f-83e2-8397d28facf8",
  "root-flag": true,
  "status": "active"
}
```

Example 2:

```
{
  "com-uuid": "068d8c73-9f83-4ac7-9fd0-84b8ba6e80da",
  "fail-reason": "",
  "node-uuid": "ff200 942-73a2-401f-83e2-8397d28facf8",
  "root-flag": true,
  "status": "waiting"
}
```

Example 3:

```
{
  "com-uuid": "9a1118bb-3b50-4d74-b9e3-deb2c33 078f8",
  "fail-reason": "",
  "node-uuid": "ff200 942-73a2-401f-83e2-8397d28facf8",
  "root-flag": true,
  "status": "running"
}
```

Service Communication

Service Communication

Source page: https://api-docs.waraps.org/service_communication/service_communication.md

?> _TODO_*Add General information related to service communication*

Some design decisions:

• Services send regular heartbeat messages with some information on a topic

• Sending a heartbeat message allows for registering the availability of the service to the rest of the system.

• Communication requiring a response should be asynchronous

• Use UUID to identify a communication message

• Unreliable communication is handled by resending commands with the same UUID until we give up or a response has arrived.

Topics

Add description on which topics are neccessary dependning on agent level e.g. sensorInfo, sensor ect.

Basic Naming Conventions

JSON

__Naming scheme__ PREFIX1/PREFIX2/PREFIX3/PREFIX4/SERVICE/#

__For sensors:__ PREFIX1/PREFIX2/PREFIX3/PREFIX4/SERVICE/sensor/SENSOR

__For heartbeat/info messages:__ PREFIX1/PREFIX2/PREFIX3/PREFIX4/SERVICE/XXX

>• *PREFIX1:* naming of context, can be used to have parallel experiments/demos. E.g: waraps-demo-2021, waraps, aiics-testing, and so on.

• *PREFIX2:* In this case service to indicate that it is a service that have heartbeat.

• *PREFIX3:* type of service: virtual.

• *PREFIX4:* Indicate role. Possible values: real, simulation, playback...

• *SERVICE:* Service name

• *SENSOR:* Name or reference to variables published by service.

Checkout Agent Communication (api_spec/agent_communication.md) on how to publish Heartbeat (api_spec/agent_communication.md#heartbeat), SensorInfo

(api_spec/agent_communication.md#sensorinfo), Sensors (api_spec/agent_communication.md#sensors) and Direct Execution Info (api_spec/agent_communication.md#directexecutioninfo), where UNIT is changed to SERVICE.

Resource Pool

Available tasks

Request Team Leader

This command is sent to the Resource Pool and is a request for a suitable and available Team Leader given one or more roles a Team Leader can take on in a mission (based on capabilities). The request should be published on the topic .../name of resource pool service/exec/command.

In addition to the direct execution response of the command, the Resource Pool should also publish a response, with the same UUID v.4 as in the direct execution response, but with an added name/value pair of the requested Team Leader. If the task is finished without fail this response should be published on the topic .../name of resource pool service/sensor/agents.

JSON command example

```
{

  "com-uuid": <uuid v4 of command>,
  "command": "start-task",
  "execution-unit": <name of resource pool service>,
  "sender": <name of sender of command>,
  "task": {
     "name": "request-team-leader",
     "meta": {
        "agent": <name of requested agent>,
        "root-uuid": <uuid v4 of tst root node of mission>
        "request": [
           {
              "alias": <alias for requested agent>,
              "capabilities": [
                 "start-tst",
                 <wanted capability of agent>,
                 <wanted capability of agent>,
                 ...
              ]
           },
           {
```

```
                "alias": <alias for requested agent>,
                "capabilities": [
                    "start-tst",
                    <wanted capability of agent>,
                    <wanted capability of agent>,
                    ...
                ]
            }
        ]
    }
  },
  "task-uuid": <uuid v4 of task>,
  "time_added": <epoch timestamp of command>
}
```

JSON response example

```
{
  "agent-uuid": <uuid v4 of the resource pool>,
  "com-uuid": <uuid v4 of command>,
  "agents": [
    {
    "alias": <alias of the agent>,
    "agent": <name of the agent>
    }
  ],
  "response-to": <uuid v4 of the command that triggered this response>,
  "task-uuid": <uuid of task>,
}
```

Request Agents

The purpose of the command "request-agents" is for as Team Leader to receive suitable and available Team Members for a mission. The request is a list of requested agents. An agent can be requested either by name or by capabilities. The command should be published on the topic .../name of resource pool service/exec/command.

In addition to the direct execution response of the command, the Resource Pool should also publish a response, with the same UUID v.4 as in the direct execution response, but with an added name/value pair of the requested agents. If the task is finished without fail this response should be published on the topic .../name of resource pool service/sensor/agents.

JSON command example

```
{
```

```
  "com-uuid": <uuid v4 of command>,
  "command": "start-task",
  "execution-unit": <name of resource pool service recieving task>,
  "sender": <name of sender of command>,
  "task": {
     "name": "request-agents"
     "meta": {
        "root-uuid": <uuid v4 of tst root node of mission>,
        "request": [
           {
              "agent": "",
              "alias": "<alias for requested agent>",
              "capabilities": [
                 <wanted capability of agent>,
                 <wanted capability of agent>,
                 ...
              ]
           },
           {
              "agent": <name of requested agent>,
              "alias": "",
              "capabilities": [
                 <wanted capability of agent>,
                 <wanted capability of agent>,
                 ...
              ]
           },
           ...
        ]
     }
  },
  "task-uuid": <uuid v4 of task>,
  "time_added": <epoch timestamp of command>
}
```

JSON response example

```
{
  "agent-uuid": <uuid v4 of the resource pool>,
  "com-uuid": <uuid v4 of command>,
  "agents": [
     {
     "alias": <alias of the agent>,
     "agent": <name of the agent>
     },
     {
     "alias": "",
     "agent": <name of the agent>
     },
     ...
  ],
  "response-to": <uuid v4 of the command that triggered this response>,
  "task-uuid": <uuid v4 of task>,
}
```

Object Detector (Providence)

The orchestrator (Providence) is a central component that efficiently manages and coordinates the deployment of multiple object detection modules running on individual streams. It acts as a conductor, initiating the execution of various modules, which are responsible for detecting objects in real-time video streams.

Available tasks

Start Object Detection

This command is sent to providence (the object detector service) to start an object detection on the stream from the agent which sends the command.

The Object Detector responds according to tasks specification (api_spec/agent_communication.md#tasks) in section Agent Communication (api_spec/agent_communication.md)

Supported Streams protocol:

- **rtsp**
- **rtmp**
- **http**
- **https**

JSON command example

```
{
        "com-uuid": <uuid v4 of command>,
        "command": "start-task",
        "execution-unit": "simulated_agent",
        "sender": "commander",
        "task": {
                "name": "start-object-detection",
                "params": {
                        "agent": <agent_name>,
                        "classes": "0",
                        "weights": "yolov7-e6e.pt",
                        "conf-thres": "0.5",
                        "iou-thres": "0.45",
                        "source": "rtmp://rtmp.waraps.org/live/<agent_name>",
                        "output-url": "rtmp://ome.waraps.org/app/'<agent_name>_OD'"
                },
        },
        "task-uuid": <uuid v4 of command>
}
```

JSON response example(s)

Running

```
{
  "agent-uuid": <uuid v4 of command>,
  "com-uuid": <uuid v4 of command>,
  "fail-reason": "",
  "response": "running",
  "response-to": <uuid v4 of command>,
  "task-uuid": <uuid v4 of command>
}
```

Failed

Returns failed if a detection is aleady running on that agent/stream

```
{
  "agent-uuid": <uuid v4 of command>,
  "com-uuid": <uuid v4 of command>,
  "fail-reason": "Detection already running on that stream",
  "response": "failed",
  "response-to": <uuid v4 of command>,
  "task-uuid": <uuid v4 of command>
}
```

Stop Object Detection

This command is sent to providence (the object detector service) to stop an object detection on the stream from the agent which sends the command.

JSON command example

```
{
  "com-uuid": "12e42b18-94f4-4562-b2ef-33d5 817 007dd",
  "command": "start-task",
  "execution-unit": "rk_laptop",
  "sender": "commander",
  "task": {
    "name": "stop-object-detection",
    "params": {
      "agent": <agent name>
    }
  },
  "task-uuid": "34ea828c-b922-4245-949e-27ca3f6bda42"
}
```

JSON response example(s)

Running

Returns running if the command was successful in stopping the object detector

```
{
```

```
 "agent-uuid": <uuid v4 of command>,
 "com-uuid": <uuid v4 of command>,
 "fail-reason": "",
 "response": "running",
 "response-to": <uuid v4 of command>,
 "task-uuid": <uuid v4 of command>
}
```


Failed

Returns failed if a agent/stream does not have a detection running.


```
{
 "agent-uuid": <uuid v4 of command>,
 "com-uuid": <uuid v4 of command>,
 "fail-reason": "Stream does not exist or not running a detection on that stream",
 "response": "failed",
 "response-to": <uuid v4 of command>,
 "task-uuid": <uuid v4 of command>
}
```


U-Space Service Provider (USSP)

A U-Space Service Provider (USSP) is an entity that operates and manages a U-Space system. U-Space refers to the airspace management ecosystem specifically designed for unmanned aircraft systems (UAS), commonly known as drones.

As the use of drones continues to grow, a USSP plays a crucial role in ensuring the safe and efficient integration of these unmanned vehicles into the airspace. The USSP acts as an intermediary between UAS operators and traditional airspace authorities, such as air traffic control.

Coordinate reference system (CRS): EPSG:5849

NOTE

This USSP is a **SUPER SIMPLE** example of how a USSP could work. It is not supposed to be used in real life scenarios. More of an example of how communication with an USSP can work. Alot of values and algorithms are static and only used as an examples.

Workflow

To start a communicating with the USSP, create a separate channel using the Start Communication (#start-communication) command

(Start Communication (#start-communication)) →

All other tasks should be sent on this new topic.

(Request Height (#request-height)) →

(Request Plan (#request-plan)) → (Get Plan (#get-plan)) → (Accept Plan (#accept-plan)) → (Activate Plan (#activate-plan)) → (End Plan (#end-plan) | Cancel Plan (#cancel-plan)) →

(Request Plan (#request-plan)) → (Get Plan (#get-plan)) → (End Plan (#end-plan) | Cancel Plan (#cancel-plan)) →

Available tasks

Start Communication

Creates a new topic for the entity and USSP to communicate on.

JSON command example

```
{
   "com-uuid":  <uuid v4 of command>,
   "command": "start-task",
   "execution-unit": <name of receiving unit>,
   "sender":  <name of sender of command>,
   "task": {
      "name": "start-communication",
      "meta": {},
      "params": {
         "name": "<name of requester>"
      }
   },
   "task-uuid": <uuid v4 of command>
}
```

JSON response example

```
  {
     "name": <name of requester>,
     "ussp_topic": <new topic>
  }
```

Request Height

JSON command example

```
{
   "com-uuid":  <uuid v4 of command>,
   "command": "start-task",
   "execution-unit": <name of receiving unit>,
   "sender":  <name of sender of command>,
   "task": {
      "name": "RequestPlan",
      "meta": {},
      "params": {
         "request": "query ground height"
      }
   },
   "task-uuid": <uuid v4 of command>
```

```
}
```

JSON response example

```
{
   "reply": "query ground height",
   "height": <height of ground over the Baltic Sea>,
   "response": "query ground height",
   "response-to": <uuid v4 of command>
}
```

Request Plan

Request (create) a plan

JSON command example

```
{
   "com-uuid": <uuid v4 of command>,
   "command": "start-task",
   "execution-unit": <name of receiving unit>,
   "sender": <name of sender of command>,
   "task": {
      "name": "request-plan",
      "meta": {
      },
      "params": {
         "request": "request plan",
         "operator ID": <operator ID>,
         "UAS ID": <UAS ID>,
         "EPSG": <EPSG reference>,
         "plan": <list of nodes>,
         "when": <when the plan should start>
         "preferred speed": <speed>,
         "preferred rate of ascend": <speed>,
         "preferred rate of descend": <speed>
      }
   },
   "task-uuid": <uuid v4 of command>
}
```

A array of nodes is used in "plan".

```
{
   "type": "2D path",
   "position": [ latitude, longitude ]
}
```

JSON response example

```
{
   "reply": "request plan",
   "plan ID": <plan id>,
   "delay": <delay bofore plan can start>,
   "response": "request plan",
   "response-to": <uuid v4 of command>
```

```
}
```

Get Plan

When a Plan has been requested (created) by the USSP. It can get fetched withteh "Get Plan" command.

JSON command example

```
{
  "com-uuid": <uuid v4 of command>,
  "command": "start-task",
  "execution-unit": <name of receiving unit>,
  "sender": <name of sender of command>,
  "task": {
    "name": "get-plan",
    "meta": {
    },
    "params": {
      "request": "get plan",
      "plan ID": "<plan_id>"
    }
  },
  "task-uuid": <uuid v4 of command>
}
```

JSON response example(s)

Invalid id

Invalid id can trigger from two occurrences

1. Plan id is not a requested plan. (see Request Plan (#request-plan))
2. The plan should already been started, Time past from "when" in Request Plan (#request-plan) command.

```
{
  "reply": "get plan",
  "status": "error",
  "response": "get plan",
  "response-to": <uuid v4 of command>
}
```

Error(s)

Error status can be triggerd by several reasons. Check "message" field for more details

```
{
  "reply": "get plan",
  "status": "error",
  "message": "incorrect plan description (<plan.length> < 2 nodes)",
  "response": "get plan",
  "response-to": <uuid v4 of command>
}
```

"Inccorect plan node type" can trigger for a few reasons.

3. The plan needs a minimum of 2 nodes (start and finish).
4. The wrong node type (see JSON NODE example (#json-node-example))

```
{
   "reply": "get plan",
   "status": "error",
   "message": "incorrect plan node type <node_type>",
   "response": "get plan",
   "response-to": com_uuid
}
```

Authorized

```
{
   "reply": "get plan",
   "status": "authorized",
   "plan": <plan>,
   "response": "get plan",
   "response-to": com_uuid
}
```

Accept Plan

JSON command example

```
{
   "com-uuid": <uuid v4 of command>,
   "command": "start-task",
   "execution-unit": <name of receiving unit>,
   "sender": <name of sender of command>,
   "task": {
      "name": "accept-plan",
      "meta": {
      },
      "params": {
         "request": "accept plan",
         "plan ID": "<plan_id>"
      }
   },
   "task-uuid": <uuid v4 of command>
}
```

JSON response example(s)

Invalid id

The plan has not been Authorized. The plan gets Authorized in the Get Plan (#get-plan) command.

```
{
   "reply": "accept plan",
   "status": "invalid id",
   "response": "accept plan",
```

```
  "response-to": <uuid v4 of command>
}
```

Accepted

```
{
  "reply": "accept plan",
  "status": "accepted",
  "response": "accept plan",
  "response-to": <uuid v4 of command>
}
```

Activate Plan

JSON command example

```
{
  "com-uuid": <uuid v4 of command>,
  "command": "start-task",
  "execution-unit": <name of receiving unit>,
  "sender": <name of sender of command>,
  "task": {
     "name": "activate-plan",
     "meta": {
     },
     "params": {
        "request": "activate plan",
        "plan ID": "<plan_id>"
     }
  },
  "task-uuid": <uuid v4 of command>
}
```

JSON response example(s)

Invalid id

The plan has not been Authorized. The plan gets Authorized in the Get Plan (#get-plan) command.

```
{
  "reply": "activate plan",
  "status": "invalid id",
  "response": "activate plan",
  "response-to": <uuid v4 of command>
}
```

Activated

```
{
  "reply": "activate plan",
  "status": "activated",
  "response": "activate plan",
  "response-to": <uuid v4 of command>
}
```

End Plan

JSON command example

```
{
   "com-uuid": <uuid v4 of command>,
   "command": "start-task",
   "execution-unit": <name of receiving unit>,
   "sender": <name of sender of command>,
   "task": {
      "name": "end-plan",
      "meta": {
      },
      "params": {
         "request": "end plan",
         "plan ID": "<plan_id>"
      }
   },
   "task-uuid": <uuid v4 of command>
}
```

JSON response examples

Invalid id

The plan has not been Authorized. The plan gets Authorized in the Get Plan (#get-plan) command.

```
{
   "reply": "end plan",
   "status": "invalid id",
   "response": "end plan",
   "response-to": <uuid v4 of command>
}
```

Activated

```
{
   "reply": "end plan",
   "status": "ended",
   "response": "end plan",
   "response-to": <uuid v4 of command>
}
```

Cancel Plan

JSON command example

```
{
   "com-uuid": <uuid v4 of command>,
   "command": "start-task",
   "execution-unit": <name of receiving unit>,
   "sender": <name of sender of command>,
   "task": {
      "name": "cancel-plan",
```

```
    "meta": {
    },
    "params": {
        "request": "cancel plan",
        "plan ID": "<plan_id>"
    }
  },
  "task-uuid": <uuid v4 of command>
}
```

JSON response example(s)

Invalid id

The plan has not been Authorized. The plan gets Authorized in the Get Plan (#get-plan) command.

```
{
  "reply": "end plan",
  "status": "invalid id",
  "response": "end plan",
  "response-to": <uuid v4 of command>
}
```

Cancelled

```
{
  "reply": "cancel plan",
  "status": "cancelled",
  "response": "cancel plan",
  "response-to": <uuid v4 of command>,
}
```